

Introduction

PHP is a general-purpose programming language used mostly for web development. Created by Rasmus Lerdorf in 1994, it allows developers to create dynamic web pages with ease. For instance, developers can create a form in HTML and process it using PHP. Depending on the inputs entered into the form, developers can use PHP to display different outputs to users. Most of the time, PHP is used as a server-side language. This means that PHP code is not processed on the user's computer (also known as a client). In other words, when you access a PHP page on your browser, the code is not processed on your computer. Instead, your browser sends a request to a web server, which then processes the code and returns the result to the browser in the form of a web page. More often than not, this web server is a remote computer where the PHP files are stored. For the web server to process PHP code, a special software known as the PHP interpreter needs to be installed.

MySQL

MySQL is a relational database management system based on the Structured Query Language, which is the popular language for accessing and managing the records in the database. MySQL is open-source and free software under the GNU license. It is supported by Oracle Company. MySQL is currently the most popular database management system software used for managing the relational database. It is fast, scalable, and easy to use database management system in comparison with Microsoft SQL Server and Oracle Database. It is commonly used in conjunction with PHP scripts for creating powerful and dynamic server-side or web-based enterprise applications.

MariaDB: The MySQL Clone

After Oracle purchased Sun Microsystems (the owners of MySQL), the community became wary that MySQL might not remain fully open source, so MariaDB was forked from it to keep it free under the GNU GPL. Development of MariaDB is led by some of the original developers of MySQL, and it retains exceedingly close compatibility with MySQL. Therefore, you may well encounter MariaDB on some servers in place of MySQL.

Incorporating PHP Within HTML

By default, PHP documents end with the extension .php. When a web server encounters this extension in a requested file, it automatically passes it to the PHP processor. Of course, web servers are highly configurable, and some web developers choose to force files ending with

.htm or .html to also get parsed by the PHP processor, usually because they want to hide their use of PHP.

Your PHP program is responsible for passing back a clean file suitable for display in a web browser. At its very simple, a PHP document will output only HTML. To prove this, you can take any normal HTML document and save it as a PHP document (for example, saving index.html as index.php), and it will display identically to the original.

To trigger the PHP commands, you need to learn a new tag. Here is the first part:

```
<?php
```

The first thing you may notice is that the tag has not been closed. This is because entire sections of PHP can be placed inside this tag, and they finish only when the closing part is encountered, which looks like this:

```
?>
```

A small PHP “Hello World” program might look like;

```
<?php
    echo "Hello world";
?>
```

Use of this tag can be quite flexible. Some programmers open the tag at the start of a document and close it right at the end, outputting any HTML directly from PHP commands. Others, however, choose to insert only the smallest possible fragments of PHP within these tags wherever dynamic scripting is required, leaving the rest of the document in standard HTML. The latter type of programmer generally argues that their style of coding results in faster code, while the former says that the speed increase is so minimal that it doesn’t justify the additional complexity of dropping in and out of PHP many times in a single document. As you learn more, you will surely discover your preferred style of PHP development.

By the way, there is a slight variation to the PHP syntax. If you browse the internet for PHP examples, you may also encounter code where the opening and closing syntax looks like this:

```
<?
    echo "Hello world";
?>
```

Although it's not as obvious that the PHP parser is being called, this is a valid, alternative syntax that also usually works. But I discourage its use, as it is incompatible with XML and is now deprecated (meaning that it is no longer recommended and support could be removed in future versions).

***Note:** If you have only PHP code in a file, you may omit the closing `?>`. This can be a good practice, as it will ensure that you have no excess whitespace leaking from your PHP files (especially important when you're writing object-oriented code).*

The Structure of PHP

Using Comments

There are two ways in which you can add comments to your PHP code. The first turns a single line into a comment by preceding it with a pair of forward slashes:

```
// This is a comment
```

This version of the comment feature is a great way to temporarily remove a line of code from a program that is giving you errors. For example, you could use such a comment to hide a debugging line of code until you need it, like this:

```
// echo "x equals $x";
```

You can also use this type of comment directly after a line of code to describe its action, like this:

```
$x += 10; // Increment $x by 10
```

When you need to use multiple lines, there's a second type of comment, which looks like

```
<?php
/* This is a section
of multiline comments
which will not be
interpreted */
?>
```

You can use the `/*` and `*/` pairs of characters to open and close comments almost anywhere you like inside your code. Most, if not all, programmers use this construct to temporarily

comment out entire sections of code that do not work or that, for one reason or another, they do not wish to be interpreted.

***Note:** A common error is to use `/*` and `*/` to comment out a large section of code that already contains a commented-out section that uses those characters. You can't nest comments this way; the PHP interpreter won't know where a comment ends and will display an error message. However, if you use an editor or IDE with syntax highlighting, this type of error is easier to spot.*

Basic Syntax

PHP is quite a simple language with roots in C and Perl (if you have ever come across these), yet it looks more like Java. It is also very flexible, but there are a few rules that you need to learn about its syntax and structure.

Semicolons

You may have noticed in the previous examples that the PHP commands ended with a semicolon, like this:

```
$x += 10;
```

One of the most common causes of errors you will encounter with PHP is forgetting this semicolon. This causes PHP to treat multiple statements like one statement, which it is unable to understand, prompting it to produce a *Parse error message*.

The \$ symbol

The \$ symbol has come to be used in many different ways by different programming languages. For example, in the BASIC language, it was used to terminate variable names to denote them as strings.

In PHP, however, you must place a \$ in front of all variables. This is required to make the PHP parser faster, as it instantly knows whenever it comes across a variable.

Whether your variables are numbers, strings, or arrays, they should all look something like those shown below:

```
<?php
$mycounter = 1;
$string = "Hello";
```

```
$myarray = array("One", "Two", "Three");  
?>
```

And really that's pretty much all the syntax that you have to remember. Unlike languages such as Python, which are very strict about how you indent and lay out your code, PHP leaves you completely free to use (or not use) all the indenting and spacing you like. In fact, sensible use of whitespace is generally encouraged (along with comprehensive commenting) to help you understand your code when you come back to it. It also helps other programmers when they have to maintain your code.

PHP Variables

Variables are used to store data, like string of text, numbers, etc. Variable values can change over the course of a script. Here're some important things to know about variables:

- In PHP, a variable does not need to be declared before adding a value to it. PHP automatically converts the variable to the correct data type, depending on its value.
- After declaring a variable, it can be reused throughout the code.
- The assignment operator (=) used to assign value to a variable.

In PHP variable can be declared as: `$var_name = value;`

```
<?php  
// Declaring variables  
$msg = "Hello World!";  
$sum = 10;  
?>
```

Variable-naming rules

When creating PHP variables, you must follow these four rules:

1. Variable names, after the dollar sign, must start with a letter of the alphabet or the _ (underscore) character.
2. Variable names can contain only the characters a–z, A–Z, 0–9, and _ (underscore)

3. Variable names may not contain spaces. If a variable name must comprise more than one word, a good idea is to separate the words with the _ (underscore) character (e.g., \$user_name).
4. Variable names are case-sensitive. The variable \$High_Score is not the same as the variable \$high_score.

PHP Constants

A constant is a name or an identifier for a fixed value. Constants are like variables, except that once they are defined, they cannot be undefined or changed (except magic constants). Constants are very useful for storing data that doesn't change while the script is running. Common examples of such data include configuration settings such as database username and password, website's base URL, company name, etc.

Constants are defined using PHP's `define()` function, which accepts two arguments: the name of the constant, and its value. Once defined the constant value can be accessed at any time just by referring to its name. Below is a simple example:

```
<?php
// Defining constant
define("SITE_URL", "https://www.fpm.edu.ng");
// Using constant
echo 'Thank you for visiting - ' . SITE_URL;
?>
```

Naming Conventions / Rules for PHP Constants

Name of constants must follow the same rules as variable names, which means a valid constant name must start with a letter or underscore, followed by any number of letters, numbers or underscores with one **exception**: the \$ prefix is not required for constant names.

The PHP echo Statement

The echo statement can output one or more strings. In general terms, the echo statement can display anything that can be displayed to the browser, such as string, numbers, variables values, the results of expressions etc.

Since echo is a language construct not actually a function (like if statement), you can use it without parentheses e.g. echo or echo(). However, if you want to pass more than one parameter to echo, the parameters must not be enclosed within parentheses.

Displaying Strings of Text

The following example will show you how to display a string of text with the echo statement:

```
<?php
// Displaying string of text
echo "Hello World!";
?>
```

Displaying HTML Code

The following example will show you how to display HTML code using the echo statement:

```
<?php
// Displaying HTML code
echo "<h4>This is a simple heading.</h4>";
echo "<h4 style='color: red;'>This is heading with style.</h4>";
?>
```

Display Variables

```
<?php
// Defining variables
$msg = "Hello World!";
$number = 123456789;
$colors = array("Red", "Green", "Blue");
// Displaying variables
echo $msg;
echo "<br>";
echo $number;
echo "<br>";
echo $colors[0];
?>
```

The PHP print Statement

You can also use the print statement (an alternative to echo) to display output to the browser. Like echo the print is also a language construct not a real function. So you can also use it without parentheses like: print or print().

Both echo and print statement work exactly the same way except that the print statement can only output one string, and always returns 1. That's why the echo statement considered marginally faster than the print statement since it doesn't return any value.

PHP Data Types

The values assigned to a PHP variable may be of different data types including simple string and numeric types to more complex data types like arrays and objects.

PHP supports total eight primitive data types: Integer, Floating point number or Float, String, Booleans, Array, Object, resource and NULL. These data types are used to construct variables.

PHP Strings

A string is a sequence of letters, numbers, special characters and arithmetic values or combination of all. The simplest way to create a string is to enclose the string literal (i.e. string characters) in single quotation marks ('), like this:

```
$msg = 'Hello World';
```

You can also use double quotation marks ("). However, single and double quotation marks work in different ways. Strings enclosed in single-quotes are treated almost literally, whereas the strings delimited by the double quotes replaces variables with the string representations of their values as well as specially interpreting certain escape sequences.

The escape-sequence replacements are:

- \n is replaced by the newline character
- \r is replaced by the carriage-return character
- \t is replaced by the tab character
- \\$ is replaced by the dollar sign itself (\$)
- \" is replaced by a single double-quote (")
- \\ is replaced by a single backslash (\)

Here's an example to clarify the differences between single and double quoted strings:

```
<?php
$my_str = 'World';
echo "Hello, $my_str!<br>";      // Displays: Hello World!
echo 'Hello, $my_str!<br>';      // Displays: Hello, $my_str!
echo '<pre>Hello\tWorld!</pre>'; // Displays: Hello\tWorld!
echo "<pre>Hello\tWorld!</pre>"; // Displays: Hello  World!
echo 'I\'ll be back';           // Displays: I'll be back
?>
```

Manipulating PHP Strings

PHP provides many built-in functions for manipulating strings like calculating the length of a string, find substrings or characters, replacing part of a string with different characters, take a string apart, and many others. Here are the examples of some of these functions.

Calculating the Length of a String

The **strlen()** function is used to calculate the number of characters inside a string. It also includes the blank spaces inside the string.

```
<?php
$msg = 'Hello World';
echo strlen($msg);
?>
```

Counting Number of Words in a String

The **str_word_count()** function counts the number of words in a string.

```
<?php
$msg = 'The quick brown fox jumps over the lazy dog.';
echo str_word_count($msg);
?>
```

Replacing Text within Strings

The **str_replace()** replaces all occurrences of the search text within the target string.

```
<?php
$msg = 'If the facts do not fit the theory, change the facts.';
echo str_replace("facts", "truth", $msg);
?>
```

You can optionally pass the fourth argument to the **str_replace()** function to know how many times the string replacements was performed, like this.

```
<?php
$msg = 'If the facts do not fit the theory, change the facts.';
str_replace("facts", "truth", $msg, $count);
echo "The text was replaced $count times.";
?>
```

Operators

Operators let you specify mathematical operations to perform, such as addition, subtraction, multiplication, and division. But several other types of operators exist too, such as the string, comparison, and logical operators. Math in PHP looks a lot like plain arithmetic for instance, the following statement outputs 8:

```
echo 6 + 2;
```

Arithmetic operators

Arithmetic operators do what you would expect; they are used to perform mathematics. You can use them for the main four operations (add, subtract, multiply, and divide) as well as to find a modulus (the remainder after a division) and to increment or decrement a value (see Table 3-1)

Table 3-1. Arithmetic operators

Operator	Description	Example
+	Addition	<code>\$j + 1</code>
-	Subtraction	<code>\$j - 6</code>
*	Multiplication	<code>\$j * 11</code>
/	Division	<code>\$j / 4</code>
%	Modulus (the remainder after a division is performed)	<code>\$j % 9</code>
++	Increment	<code>++\$j</code>
--	Decrement	<code>--\$j</code>
**	Exponentiation (or power)	<code>\$j**2</code>

Assignment operators

These operators assign values to variables. They start with the very simple `=` and move on to `+=`, `-=`, and so on (see Table 3-2). The operator `+=` adds the value on the right side to the variable on the left, instead of totally replacing the value on the left. Thus, if `$count` starts with the value 5, the statement:

```
$count += 1;
```

sets `$count` to 6, just like the more familiar assignment statement:

```
$count = $count + 1;
```

The `/=` and `*=` operators are similar, but for division and multiplication, the `.=` operator concatenates variables, such that `$a .= "."` will append a period to the end of `$a`, and `%=` assigns a percentage value.

Table 3-2. Assignment operators

Operator	Example	Equivalent to
=	<code>\$j = 15</code>	<code>\$j = 15</code>
+=	<code>\$j += 5</code>	<code>\$j = \$j + 5</code>
-=	<code>\$j -= 3</code>	<code>\$j = \$j - 3</code>
*=	<code>\$j *= 8</code>	<code>\$j = \$j * 8</code>
/=	<code>\$j /= 16</code>	<code>\$j = \$j / 16</code>
.=	<code>\$j .= \$k</code>	<code>\$j = \$j . \$k</code>
%=	<code>\$j %= 4</code>	<code>\$j = \$j % 4</code>

Comparison operators

Comparison operators are generally used inside a construct such as an if statement in which you need to compare two items. For example, you may wish to know whether a variable you have been incrementing has reached a specific value, or whether another variable is less than a set value, and so on (see Table 3-3).

Table 3-3. Comparison operators

Operator	Description	Example
<code>==</code>	<i>Is equal to</i>	<code>\$j == 4</code>
<code>!=</code>	<i>Is not equal to</i>	<code>\$j != 21</code>
<code>></code>	<i>Is greater than</i>	<code>\$j > 3</code>
<code><</code>	<i>Is less than</i>	<code>\$j < 100</code>
<code>>=</code>	<i>Is greater than or equal to</i>	<code>\$j >= 15</code>
<code><=</code>	<i>Is less than or equal to</i>	<code>\$j <= 8</code>
<code><></code>	<i>Is not equal to</i>	<code>\$j <> 23</code>
<code>===</code>	<i>Is identical to</i>	<code>\$j === "987"</code>
<code>!==</code>	<i>Is not identical to</i>	<code>\$j !== "1.2e3"</code>

Note the difference between = and ==. The first is an assignment operator, and the second is a comparison operator. Even advanced programmers can sometimes mix up the two when coding hurriedly, so be careful.

Logical operators

If you haven't used them before, logical operators may at first seem a little daunting. But just think of them the way you would use logic in English. For example, you might say to yourself, "If the time is later than 12 p.m. and earlier than 2 p.m., have lunch." In PHP, the code for this might look something like the following (using military time):

```
if ($hour > 12 && $hour < 14) dolunch();
```

Here we have moved the set of instructions for actually going to lunch into a function that we will have to create later called `dolunch`.

As the previous example shows, you generally use a logical operator to combine the results of two of the comparison operators shown in the previous section. A logical operator can also be input to another logical operator: "If the time is later than 12p.m. and earlier than 2 p.m., or if the smell of a roast is permeating the hallway and there are plates on the table." As a rule, if something has a TRUE or FALSE value, it can be input to a logical operator. A logical operator takes two inputs true or false and produces a true or false result.

Table 3-4. Logical operators

Operator	Description	Example
<code>&&</code>	<i>And</i>	<code>\$j == 3 && \$k == 2</code>
<code>and</code>	Low-precedence <i>and</i>	<code>\$j == 3 and \$k == 2</code>
<code> </code>	<i>Or</i>	<code>\$j < 5 \$j > 10</code>
<code>or</code>	Low-precedence <i>or</i>	<code>\$j < 5 or \$j > 10</code>
<code>!</code>	<i>Not</i>	<code>! (\$j == \$k)</code>
<code>xor</code>	<i>Exclusive or</i>	<code>\$j xor \$k</code>

Note that `&&` is usually interchangeable with `AND`; the same is true for `||` and `OR`. However, because `AND` and `OR` have a lower precedence, you should avoid using them except when they are the only option, as in the following statement, which must use the `OR` operator (`||` cannot be used to force a second statement to execute if the first fails):

```
$html = file_get_contents($site) or die("Cannot download from $site");
```

The most unusual of these operators is xor, which stands for exclusive or and returns a TRUE value if either value is TRUE but a FALSE value if both inputs are TRUE or both inputs are FALSE. To understand this, imagine that you want to concoct your own cleaner for household items. Ammonia makes a good cleaner, and so does bleach, so you want your cleaner to have one of these. But the cleaner must not have both, because the combination is hazardous. In PHP, you could represent this as follows:

```
$ingredient = $ammonia xor $bleach;
```

In this example, if either \$ammonia or \$bleach is TRUE, \$ingredient will also be set to TRUE. But if both are TRUE or both are FALSE, \$ingredient will be set to FALSE.

Variable Assignment

The syntax to assign a value to a variable is always variable = value. Or, to reassign the value to another variable, it is other_variable = variable.

There are also a couple of other assignment operators that you will find useful. For example, we've already seen this:

```
$x += 10;
```

which tells the PHP parser to add the value on the right (in this instance, the value 10) to the variable \$x. Likewise, we could subtract as follows:

```
$y -= 10;
```

Variable incrementing and decrementing

Adding or subtracting 1 is such a common operation that PHP provides special operators for it. You can use one of the following in place of the += and -= operators:

```
++$x;
```

```
--$y;
```

PHP Spaceship Operator

The spaceship operator (<=>) was introduced in PHP 7 which can be used for comparing two expressions. It is also known as combined comparison operator.

The spaceship operator returns 0 if both operands are equal, 1 if the left is greater, and -1 if the right is greater. It basically provides three-way comparison as shown in the following table:

Operator	<=> Equivalent
<code>\$x < \$y</code>	<code>(\$x <=> \$y) === -1</code>
<code>\$x <= \$y</code>	<code>(\$x <=> \$y) === -1 (\$x <=> \$y) === 0</code>
<code>\$x == \$y</code>	<code>(\$x <=> \$y) === 0</code>
<code>\$x != \$y</code>	<code>(\$x <=> \$y) !== 0</code>
<code>\$x >= \$y</code>	<code>(\$x <=> \$y) === 1 (\$x <=> \$y) === 0</code>
<code>\$x > \$y</code>	<code>(\$x <=> \$y) === 1</code>

The following example shows you how spaceship operator actually works:

```
<?php
// Comparing Integers
echo 1 <=> 1; // Outputs: 0
echo 1 <=> 2; // Outputs: -1
echo 2 <=> 1; // Outputs: 1
// Comparing Floats
echo 1.5 <=> 1.5; // Outputs: 0
echo 1.5 <=> 2.5; // Outputs: -1
echo 2.5 <=> 1.5; // Outputs: 1
// Comparing Strings
echo "x" <=> "x"; // Outputs: 0
echo "x" <=> "y"; // Outputs: -1
echo "y" <=> "x"; // Outputs: 1
?>
```

PHP Conditional Statements

Conditionals alter program flow. They enable you to ask questions about certain things and respond to the answers you get in different ways. Conditionals are central to creating dynamic web pages, which is the goal of using PHP in the first place, because they make it easy to render different output each time a page is viewed.

Like most programming languages, PHP also allows you to write code that perform different actions based on the results of a logical or comparative test condition at run time. This means, you can create test conditions in the form of expressions that evaluates to either true or false and based on these results you can perform certain actions.

There are several statements in PHP that you can use to make decisions:

- The if statement
- The if...else statement
- The if...elseif....else statement
- The switch...case statement

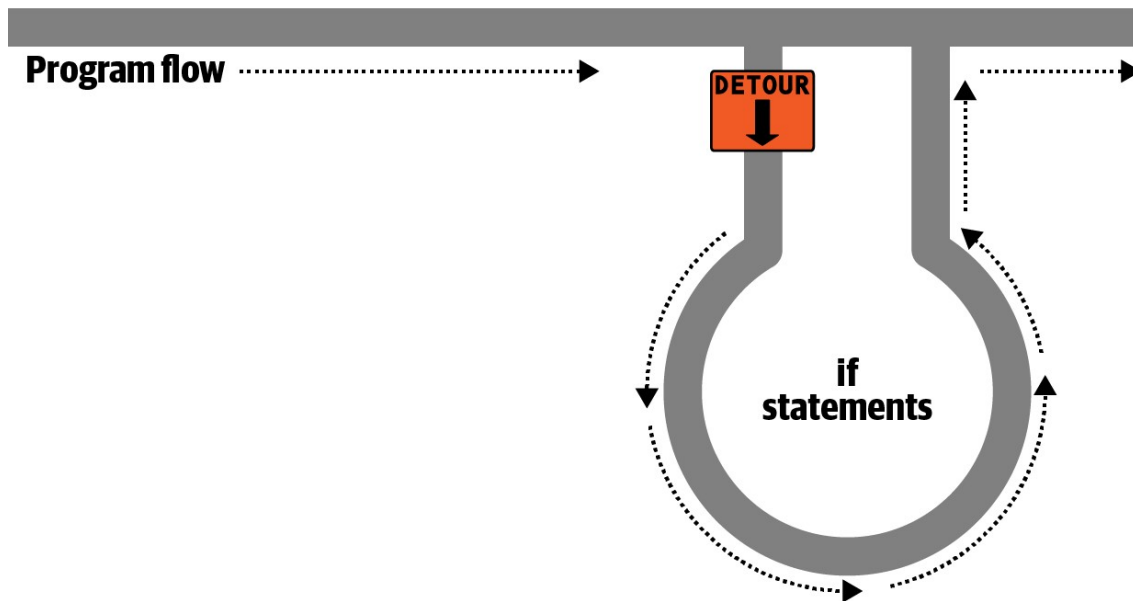
The if Statement

The **if** statement is used to execute a block of code only if the specified condition evaluates to true. This is the simplest PHP's conditional statements and can be written like:

```
if(condition){  
    // Code to be executed  
}
```

One way of thinking about program flow is to imagine it as a single-lane highway that you are driving along. It's pretty much a straight line, but now and then you encounter various signs telling you where to go.

In the case of an if statement, you could imagine coming across a detour sign that you have to follow if a certain condition is TRUE. If so, you drive off and follow the detour until you return to the main road and then continue on your way in your original direction. Or, if the condition isn't TRUE, you ignore the detour and carry on driving as shown below;



The contents of the if condition can be any valid PHP expression, including tests for equality, comparison expressions, tests for 0 and NULL, and even functions (either built-in functions or ones that you write).

The actions to take when an if condition is TRUE are generally placed inside curly braces ({ }). You can ignore the braces if you have only a single statement to execute, but if you always use curly braces, you'll avoid having to hunt down difficult-to-trace bugs, such as when you add an extra line to a condition and it doesn't get evaluated due to the lack of braces.

The following example will output "Have a nice weekend!" if the current day is Friday:

```
<?php
    $d = date("D");
    if($d == "Fri"){
        echo "Have a nice weekend!";
    }
?>
```

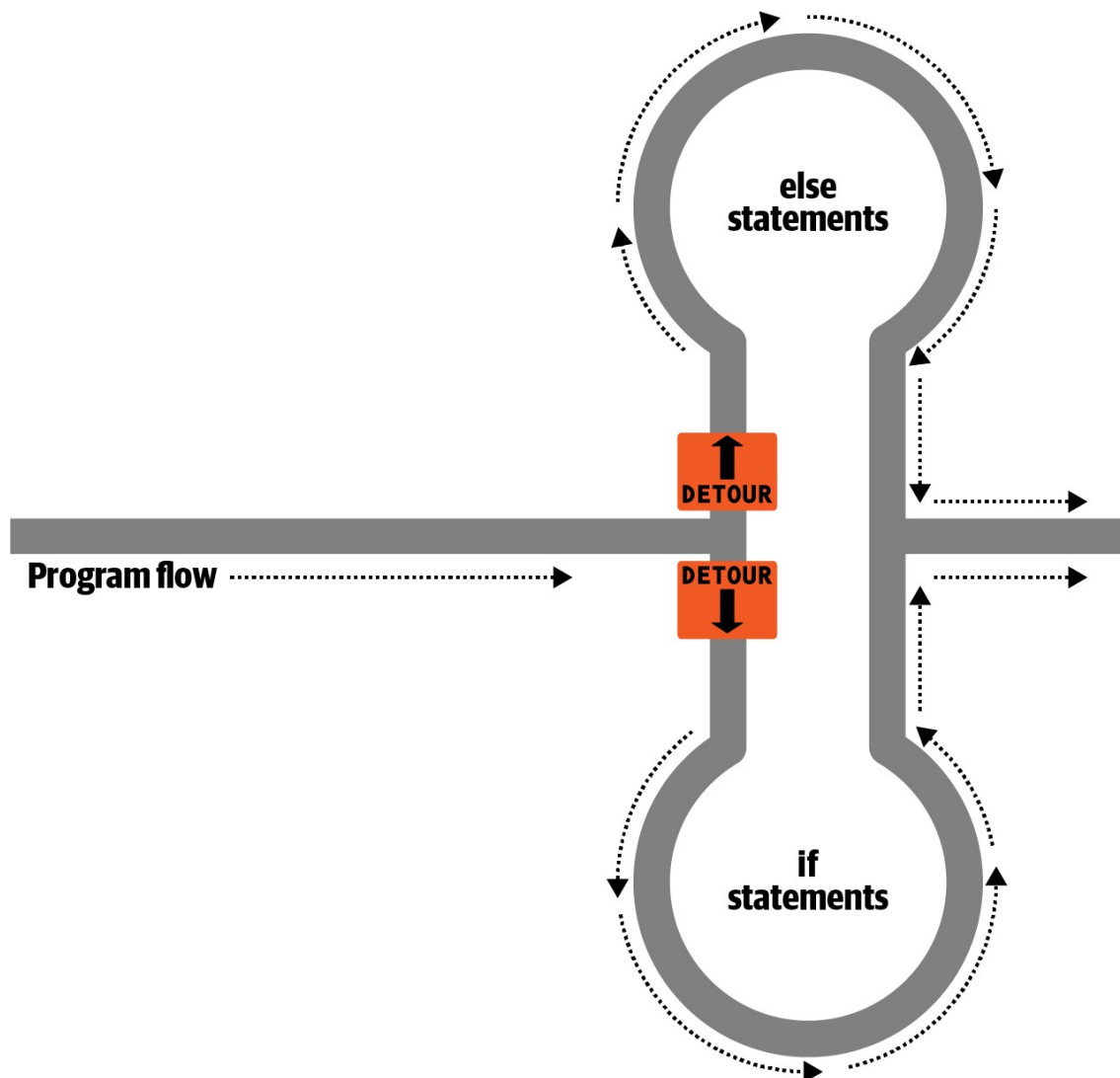
The if...else Statement

You can enhance the decision-making process by providing an alternative choice through adding an else statement to the if statement. The if...else statement allows you to execute one

block of code if the specified condition is evaluates to true and another block of code if it is evaluates to false. It can be written, like this:

```
if(condition){  
    // Code to be executed if condition is true  
} else{  
    // Code to be executed if condition is false  
}
```

Sometimes when a conditional is not TRUE, you may not want to continue on to the main program code immediately but might wish to do something else instead. This is where the else statement comes in. With it, you can set up a second detour on your highway, as shown below.



With an if...else statement, the first conditional statement is executed if the condition is TRUE. But if it's FALSE, the second one is executed. One of the two choices must be executed. Under no circumstance can both (or neither) be executed.

The following example will output "Have a nice weekend!" if the current day is Friday, otherwise it will output "Have a nice day!"

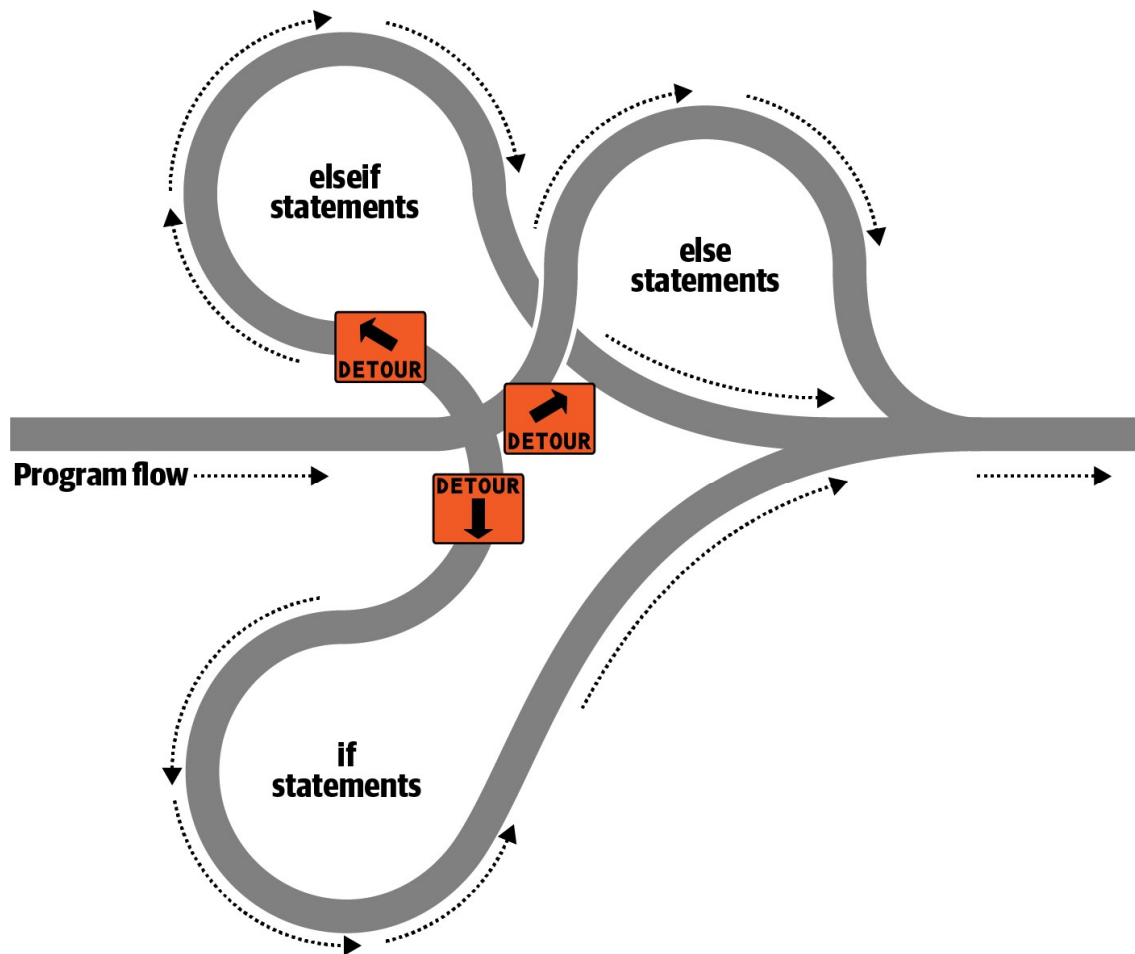
```
<?php
    $d = date("D");
    if($d == "Fri"){
        echo "Have a nice weekend!";
    } else{
        echo "Have a nice day!";
    }
?>
```

The if...elseif...else Statement

The if...elseif...else is a special statement that is used to combine multiple if...else statements.

```
if(condition1){
    // Code to be executed if condition1 is true
} elseif(condition2){
    // Code to be executed if the condition1 is false and
    condition2 is true
} else{
    // Code to be executed if both condition1 and condition2
    are false
}
```

There are also times when you want a number of different possibilities to occur, based upon a sequence of conditions. You can achieve this using the elseif statement. As you might imagine, it is like an else statement, except that you place a further conditional expression prior to the conditional code.



You may have as many elseif statements as you like. But as the number of elseif statements increases, you would probably be better advised to consider a switch statement if it fits your needs.

Note: An **else** statement closes either an *if...else* or an *if...elseif...else* statement. You can leave out a final else if it is not required, but you cannot have one before an elseif; you also cannot have an elseif before an if statement.

The following example will output "Have a nice weekend!" if the current day is Friday, and "Have a nice Sunday!" if the current day is Sunday, otherwise it will output "Have a nice day!"

```
<?php
    $d = date("D");
    if($d == "Fri"){
        echo "Have a nice weekend!";
```

```

    } elseif($d == "Sun"){
        echo "Have a nice Sunday!";
    } else{
        echo "Have a nice day!";
    }
?>

```

The ? (or ternary) Operator

One way of avoiding the verbosity of if and else statements is to use the more compact ternary operator, **?**, which is unusual in that it takes three operands rather than the typical two. The **?** operator is passed an expression that it must evaluate, along with two statements to execute: one for when the expression evaluates to TRUE, the other for when it is FALSE.

The ternary operator provides a shorthand way of writing the **if...else** statements. The ternary operator is represented by the question mark (**?**) symbol and it takes three operands: a condition to check, a result for true, and a result for false.

To understand how this operator works, consider the following examples:

```

<?php
if($age < 18){
    echo 'Child'; // Display Child if age is less than 18
} else{
    echo 'Adult'; // Display Adult if age is greater than or equal
to 18
}
?>

```

Using the ternary operator, the same code could be written in a more compact way:

```

<?php
echo ($age < 18) ? 'Child' : 'Adult';
?>

```

The ternary operator in the example above selects the value on the left of the colon (i.e. 'Child') if the condition evaluates to true (i.e. if \$age is less than 18), and selects the value on the right of the colon (i.e. 'Adult') if the condition evaluates to false.

***Note:** Code written using the ternary operator can be hard to read. However, it provides a great way to write compact if-else statements.*

The Null Coalescing Operator (PHP >= 7)

PHP 7 introduces a new null coalescing operator (??) which you can use as a shorthand where you need to use a ternary operator in conjunction with isset() function.

To understand this in a better way consider the following line of code. It fetches the value of `$_GET['name']`, if it does not exist or NULL, it returns 'anonymous'.

```
<?php
$name = isset($_GET['name']) ? $_GET['name'] : 'anonymous';
?>
```

Using the null coalescing operator, the same code could be written as:

```
<?php
$name = $_GET['name'] ?? 'anonymous';
?>
```

The switch Statement

The **switch** statement is useful where one variable, or the result of an expression, can have multiple values, each of which should trigger a different activity.

PHP If...Else Vs Switch...Case

The switch-case statement is an alternative to the if-elseif-else statement, which does almost the same thing. The switch-case statement tests a variable against a series of values until it finds a match, and then executes the block of code corresponding to that match.

```
switch(n){
    case label1:
        // Code to be executed if n=label1
        break;
    case label2:
        // Code to be executed if n=label2
        break;
    ...
}
```

```
        default:
            // Code to be executed if n is different from all
labels
    }
```

Breaking out

If you wish to break out of the switch statement because a condition has been fulfilled, use the break command. This command tells PHP to exit the switch and jump to the following statement.

Default action

A typical requirement in switch statements is to fall back on a default action if none of the case conditions are met.

```
<?php
$today = date("D");
switch($today){
    case "Mon":
        echo "Today is Monday. Clean your house.";
        break;
    case "Tue":
        echo "Today is Tuesday. Buy some food.";
        break;
    case "Wed":
        echo "Today is Wednesday. Visit a doctor.";
        break;
    case "Thu":
        echo "Today is Thursday. Repair your car.";
        break;
    case "Fri":
        echo "Today is Friday. Party tonight.";
        break;
    case "Sat":
```

```

        echo "Today is Saturday. Its movie time.";
        break;
    case "Sun":
        echo "Today is Sunday. Do some rest.";
        break;
    default:
        echo "No information available for that day.";
        break;
}
?>

```

Note: The *switch-case* statement differs from the *if-elseif-else* statement in one important way. The *switch* statement executes line by line (i.e. statement by statement) and once PHP finds a case statement that evaluates to true, it's not only executes the code corresponding to that case statement, but also executes all the subsequent case statements till the end of the switch block automatically.

To prevent this, add a *break* statement to the end of each case block. The *break* statement tells PHP to break out of the *switch-case* statement block once it executes the code associated with the first true case.

PHP Loops / Looping

One of the great things about computers is that they can repeat calculating tasks quickly and tirelessly. Often you may want a program to repeat the same sequence of code again and again until something happens, such as a user inputting a value or reaching a natural end. PHP's loop structures provide the perfect way to do this.

The basic idea behind a loop is to automate the repetitive tasks within a program to save time and effort. PHP supports four different types of loops as follows;

1. **While** :- loops through a block of code as long as the condition specified evaluates to true.
2. **do...while** :- the block of code executed once and then condition is evaluated. If the condition is true the statement is repeated as long as the specified condition is true.

3. **for** :- loops through a block of code until the counter reaches a specified number.
4. **foreach** :- loops through a block of code for each element in an array.

while Loop

The **while** statement loops through a block of code as long as the condition specified in the while statement evaluate to true.

```
while(condition){  
    // Code to be executed  
}
```

The example below define a loop that starts with **\$i=1**. The loop will continue to run as long as **\$i** is less than or equal to **3**. The **\$i** will increase by **1** each time the loop runs:

```
<?php  
    $i = 1;  
    while($i <= 3){  
        $i++;  
        echo "The number is " . $i . "<br>";  
    }  
?>
```

do...while Loop

The **do-while** loop is a variant of while loop, which evaluates the condition at the end of each loop iteration. With a do-while loop the block of code executed once, and then the condition is evaluated, if the condition is true, the statement is repeated as long as the specified condition evaluated to is true.

The do-while loop is a variant of while loop, which evaluates the condition at the end of each loop iteration. With a do-while loop the block of code executed once, and then the condition is evaluated, if the condition is true, the statement is repeated as long as the specified condition evaluated to is true.

```
do{
    // Code to be executed
}
while(condition);
```

The following example define a loop that starts with **\$i=1**. It will then increase **\$i** with **1**, and print the output. Then the condition is evaluated, and the loop will continue to run as long as **\$i** is less than, or equal to **3**.

```
<?php
    $i = 1;
    do{
        $i++;
        echo "The number is " . $i . "<br>";
    }
    while($i <= 3);
?>
```

Difference Between while and do...while Loop

The **while** loop differs from the **do-while** loop in one important way, with a while loop, the condition to be evaluated is tested at the beginning of each loop iteration, so if the conditional expression evaluates to false, the loop will never be executed. While a **do-while** loop, on the other hand, will always be executed once, even if the conditional expression is false, because the condition is evaluated at the end of the loop iteration rather than the beginning.

for Loop

The **for** loop repeats a block of code as long as a certain condition is met. It is typically used to execute a block of code for certain number of times.

```
for(initialization; condition; increment){
    // Code to be executed
}
```

The parameters of for loop have following meanings:

initialization it is used to initialize the counter variables, and evaluated once unconditionally before the first execution of the body of the loop.

condition in the beginning of each iteration, condition is evaluated. If it evaluates to true, the loop continues and the nested statements are executed. If it evaluates to false, the execution of the loop ends.

increment it updates the loop counter with a new value. It is evaluated at the end of each iteration.

The example below defines a loop that starts with **\$i=1**. The loop will continue until **\$i** is less than, or equal to **3**. The variable **\$i** will increase by **1** each time the loop runs:

```
<?php
    for($i=1; $i<=3; $i++){
        echo "The number is " . $i . "<br>";
    }
?>
```

foreach Loop

foreach loop is used to iterate over arrays.

```
foreach($array as $value){
    // Code to be executed
}
```

The following example demonstrates a loop that will print the values of the given array:

```
<?php
$colors = array("Red", "Green", "Blue");
// Loop through colors array
foreach($colors as $value){
    echo $value . "<br>";
}
```

?>

There is one more syntax of foreach loop, which is extension of the first.

```
foreach($array as $key => $value){  
    // Code to be executed  
}  
  
<?php  
$superhero = array(  
    "name" => "Peter Parker",  
    "email" => "peterparker@mail.com",  
    "age" => 18  
);  
  
// Loop through superhero array  
foreach($superhero as $key => $value){  
    echo $key . " : " . $value . "<br>";  
}  
?>
```

PHP Functions

A function is a self-contained block of code that performs a specific task.

PHP comes with hundreds of ready-mades, **built-in functions**, making it a very rich language. To use a function, call it by name. For example, you can see the date function in action here:

```
echo date("l"); // Displays the day of the week
```

The parentheses tell PHP that you're referring to a function. Otherwise, it thinks you're referring to a constant or variable. Functions can take any number of arguments, including zero.

PHP Built-in Functions

PHP has a huge collection of **internal** or **built-in functions** that you can call directly within your PHP scripts to perform a specific task, like `gettype()`, `print_r()`, `var_dump()`, etc.

PHP User-Defined Functions

In addition to the **built-in functions**, PHP also allows you to define your own functions. It is a way of creating reusable code packages that perform specific tasks and can be kept and maintained separately from the main program.

Below are some advantages of using functions;

1. **Functions reduces the repetition of code within a program** Function allows you to extract commonly used block of code into a single component. Now you can perform the same task by calling this function wherever you want within your script without having to copy and paste the same block of code again and again.
2. **Functions makes the code much easier to maintain** Since a function created once can be used many times, so any changes made inside a function automatically implemented at all the places without touching several files.
3. **Functions makes it easier to eliminate the errors** When the program is subdivided into functions, if any error occur you know exactly what function causing the error and where to find it. Therefore, fixing errors becomes much easier.
4. **Functions can be reused in other application** Because a function is separated from the rest of the script, it's easy to reuse the same function in other applications just by including the file containing those functions.

Creating / Defining a Function

The general syntax for a function is as follows:

```
function function_name([parameter [, ...]])  
{  
    // Statements  
}
```

The first line of the syntax indicates the following:

- A definition starts with the word **function**.
- A **name** follows, which must start with a **letter** or **underscore**, followed by any number of **letters**, **numbers**, or **underscores**.
- The **parentheses** are required.
- One or more **parameters**, separated by **commas**, are **optional** (as indicated by the square brackets).

Function names are **case-insensitive**, so all of the following strings can refer to the print function: PRINT, Print, and PrInT.

The opening **curly brace** starts the statements that will execute when you call the function; a matching **curly brace** must close it. These statements may include one or more return statements, which force the function to cease execution and return to the calling code. If a value is attached to the return statement, the calling code can retrieve it.

This is a simple example of a user-defined function, that display today's date:

```
<?php

// Defining function

function whatIsToday(){

    echo "Today is " . date('l');

}

// Calling function

whatIsToday();

?>
```

Functions with Parameters

You can specify parameters when you define your function to accept input values at run time. The parameters work like placeholder variables within a function; they're replaced at run time by the values (known as argument) provided to the function at the time of invocation.

```
function myFunc($oneParameter, $anotherParameter){

    // Code to be executed

}
```

You can define as many parameters as you like. However, for each parameter you specify, a corresponding argument needs to be passed to the function when it is called.

The `getSum()` function in following example takes two integer values as arguments, simply add them together and then display the result.

```
<?php
// Defining function
function getSum($num1, $num2){
    $sum = $num1 + $num2;
    echo "Sum of the two numbers $num1 and $num2 is : $sum";
}

// Calling function
getSum(10, 20);
?>
```

Functions with Optional Parameters and Default Values

You can also create functions with optional parameters, just insert the parameter name, followed by an **equals (=)** sign, followed by a default value, as shown below;

```
<?php
// Defining function
function customFont($font, $size=1.5){
    echo "<p style=\"font-family: $font; font-size: {$size}em;\">Hello, world!</p>";
}

// Calling function
customFont("Arial", 2);
customFont("Times", 3);
customFont("Courier");
?>
```

As you can see the third call to `customFont()` doesn't include the second argument. This causes PHP engine to use the default value for the `$size` parameter which is 1.5.

Returning Values from a Function

A function can return a value back to the script that called the function using the **return** statement. The value may be of any type, including arrays and objects.

```
<?php
// Defining function
function getSum($num1, $num2){
    $total = $num1 + $num2;
    return $total;
}
// Printing returned value
echo getSum(5, 10); // Outputs: 15
?>
```

A function cannot return multiple values. However, you can obtain similar results by returning an array, as demonstrated in the following example.

```
<?php
// Defining function
function divideNumbers($dividend, $divisor){
    $quotient = $dividend / $divisor;
    $array = array($dividend, $divisor, $quotient);
    return $array;
}

// Assign variables as if they were an array
list($dividend, $divisor, $quotient) = divideNumbers(10, 2);
echo $dividend; // Outputs: 10
echo $divisor; // Outputs: 2
echo $quotient; // Outputs: 5
?>
```

Passing Arguments to a Function by Reference

In PHP there are two ways you can pass arguments to a function: by value and by reference. By default, function arguments are passed by value so that if the value of the argument within the function is changed, it does not get affected outside of the function. However, to allow a function to modify its arguments, they must be passed by reference.

Passing an argument by reference is done by prepending an **ampersand (&)** to the argument name in the function definition, as shown in the example below;

```
<?php
/* Defining a function that multiply a number
by itself and return the new value */
function selfMultiply(&$number){
    $number *= $number;
    return $number;
}

$mynum = 5;
echo $mynum; // Outputs: 5

selfMultiply($mynum);
echo $mynum; // Outputs: 25
?>
```

Understanding the Variable Scope

However, you can declare the variables anywhere in a PHP script. But, the location of the declaration determines the extent of a variable's visibility within the PHP program i.e., where the variable can be used or accessed. This accessibility is known as **variable scope**.

By default, variables declared within a function are **local** and they cannot be viewed or manipulated from outside of that function, as demonstrated in the example below:

```

<?php
// Defining function
function test(){
    $greet = "Hello World!";
    echo $greet;
}

test(); // Outputs: Hello World!

echo $greet; // Generate undefined variable error
?>

```

Similarly, if you try to access or import an outside variable inside the function, you'll get an undefined variable error, as shown in the following example;

```

<?php
$greet = "Hello World!";

// Defining function
function test(){
    echo $greet;
}

test(); // Generate undefined variable error

echo $greet; // Outputs: Hello World!
?>

```

As you can see in the above examples the variable declared inside the function is not accessible from outside, likewise the variable declared outside of the function is not accessible inside of the function. This separation reduces the chances of variables within a function getting affected by the variables in the main program.

***Note:** It is possible to reuse the same name for a variable in different functions, since local variables are only recognized by the function in which they are declared.*

The global Keyword

There may be a situation when you need to import a variable from the main program into a function, or vice versa. In such cases, you can use the global keyword before the variables inside a function. This keyword turns the variable into a global variable, making it visible or accessible both inside and outside the function, as show in the example below;

```
<?php
$greet = "Hello World!";

// Defining function
function test(){
    global $greet;
    echo $greet;
}

test(); // Outpus: Hello World!
echo $greet; // Outpus: Hello World!

// Assign a new value to variable
$greet = "Goodbye";

test(); // Outputs: Goodbye
echo $greet; // Outputs: Goodbye
?>
```

Creating Recursive Functions

A recursive function is a function that calls **itself** again and again until a condition is satisfied. Recursive functions are often used to solve complex mathematical calculations, or to process deeply nested structures e.g., printing all the elements of a deeply nested array.

The following example demonstrates how a recursive function works.

```
<?php
    function naturalNumbers($number) {
        if($number <= 10){
            echo "$number <br/>";
            naturalNumbers($number+1);
        }
    }

    naturalNumbers(1);
?>
```

```
<?php
    function factorial($n) {
        if ($n < 0) {
            return -1;
        }
        if ($n == 0) {
            return 1;
        }
        return ($n * factorial ($n -1));
    }
    echo factorial(10);
?>
```

Note: Be careful while creating recursive functions, because if code is written improperly it may result in an infinite loop of function calling.